

```

b=brr[l+1];
for (;;) {
    do i++; while (arr[i] < a);
    do j--; while (arr[j] > a);
    if (j < i) break;
    SWAP(arr[i],arr[j]);
    SWAP(brr[i],brr[j]);
}
arr[l+1]=arr[j];
arr[j]=a;
brr[l+1]=brr[j];
brr[j]=b;
jstack += 2;
Push pointers to larger subarray on stack; process smaller subarray immediately.
if (jstack >= NSTACK) throw("NSTACK too small in sort2.");
if (ir-i+1 >= j-1) {
    istack[jstack]=ir;
    istack[jstack-1]=i;
    ir=j-1;
} else {
    istack[jstack]=j-1;
    istack[jstack-1]=l;
    l=i;
}
}
}

```

Beginning of innermost loop.
 Scan up to find element > a.
 Scan down to find element < a.
 Pointers crossed. Partitioning complete.
 Exchange elements of both arrays.
 End of innermost loop.
 Insert partitioning element in both arrays.

You could, in principle, rearrange any number of additional arrays along with `brr`, but this is inefficient if the number of such arrays is larger than one. The preferred technique is to make use of an index table, as described in §8.4.

CITED REFERENCES AND FURTHER READING:

Sedgewick, R. 1978, "Implementing Quicksort Programs," *Communications of the ACM*, vol. 21, pp. 847–857.[1]

8.3 Heapsort

Heapsort is slower than Quicksort by a constant factor. It is so beautiful that we sometimes use it anyway, just for the sheer joy of it. (However, we don't recommend that you do this if your employer is paying for efficient code.) Heapsort is a true "in-place" sort, requiring no auxiliary storage. It is an $N \log_2 N$ algorithm, not only on average, but also for the worst case order of input data. In fact, its worst case is only 20 % or so worse than its average running time.

It is beyond our scope to give a complete exposition on the theory of Heapsort. We mention the general principles, then refer you to the references [1,2]; or you can analyze the program yourself, if you want to understand the details.

A set of N numbers a_j , $j = 0, \dots, N - 1$, is said to form a "heap" if it satisfies the relation

$$a_{(j-1)/2} \geq a_j \quad \text{for} \quad 0 \leq (j-1)/2 < j < N \quad (8.3.1)$$

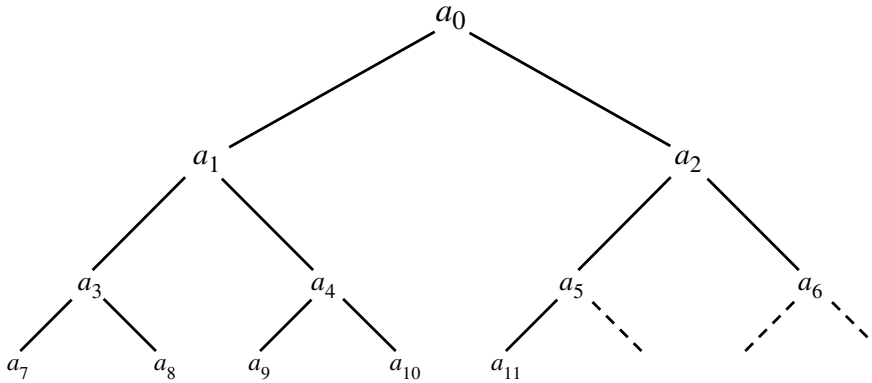


Figure 8.3.1. Ordering implied by a “heap,” here of 12 elements. Elements connected by an upward path are sorted with respect to one another, but there is not necessarily any ordering among elements related only “laterally.”

Here the division in $j/2$ means “integer divide,” i.e., is an exact integer or else is rounded down to the closest integer. Definition (8.3.1) will make sense if you think of the numbers a_i as being arranged in a binary tree, with the top, “boss,” node being a_0 ; the two “underling” nodes being a_1 and a_2 ; *their* four underling nodes being a_3 through a_6 ; etc. (See Figure 8.3.1.) In this form, a heap has every “supervisor” greater than or equal to its two “supervisees,” down through the levels of the hierarchy.

If you have managed to rearrange your array into an order that forms a heap, then sorting it is very easy: You pull off the “top of the heap,” which will be the largest element yet unsorted. Then you “promote” to the top of the heap its largest underling. Then you promote *its* largest underling, and so on. The process is like what happens (or is supposed to happen) in a large corporation when the chairman of the board retires. You then repeat the whole process by retiring the new chairman of the board. Evidently the whole thing is an $N \log_2 N$ process, since each retiring chairman leads to $\log_2 N$ promotions of underlings.

Well, how do you arrange the array into a heap in the first place? The answer is again a “sift-up” process like corporate promotion. Imagine that the corporation starts out with $N/2$ employees on the production line, but with no supervisors. Now a supervisor is hired to supervise two workers. If he is less capable than one of his workers, that one is promoted in his place, and he joins the production line. After supervisors are hired, then supervisors of supervisors are hired, and so on up the corporate ladder. Each employee is brought in at the top of the tree, but then immediately sifted down, with more capable workers promoted until their proper corporate level has been reached.

In the Heapsort implementation, the same sift-up code can be used for the initial creation of the heap and for the subsequent retirement-and-promotion phase. One execution of the Heapsort function represents the entire life-cycle of a giant corporation: $N/2$ workers are hired; $N/2$ potential supervisors are hired; there is a sifting up in the ranks, a sort of super Peter Principle: In due course, each of the original employees gets promoted to chairman of the board.